

Artículo • 06/04/2023

Expandir tabla

 $\frac{1}{4}$

Tipo de Visual Basic	Estructura de tipos de Common Language Runtime	Asignación de almacenamiento nominal	Intervalo de valores
Object	Object (clase)	4 bytes en plataforma de 32 bits 8 bytes en plataforma de 64 bits	Cualquier tipo puede almacenarse en una variable de tipo <code>Object</code> .
SByte	SByte	1 byte	De -128 a 127 (con signo).
Short (entero corto)	Int16	2 bytes	De -32 768 a 32 767 (con signo).
Single (punto flotante de precisión sencilla)	Single	4 bytes	De -3,4028235E+38 a -1,401298E-45 [†] para valores negativos; de 1,401298E-45 a 3,4028235E+38 [†] para valores positivos.
String (longitud variable)	String (clase)	Depende de la plataforma de implementación.	De 0 a 2000 millones de caracteres Unicode aproximadamente.
UInteger	UInt32	4 bytes	De 0 a 4 294 967 295 (sin signo).
ULong	UInt64	8 bytes	De 0 a 18 446 744 073 709 551 615 (1,8...E+19 [†]) (sin signo).
Definido por el usuario (estructura)	(Hereda de ValueType)	Depende de la plataforma de implementación.	Cada miembro de la estructura tiene un intervalo determinado por su tipo de datos e independiente de los intervalos de los otros miembros.
UShort	UInt16	2 bytes	De 0 a 65 535 (sin signo).

[†] En *notación científica*, "E" se refiere a una potencia de 10. Por lo tanto, 3,56E+2 significa $3,56 \times 10^2$ o 356, y 3,56E-2 significa $3,56 / 10^2$ o 0,0356.

📌 Nota

Para las cadenas que contienen texto, use la función [StrConv](#) para realizar la conversión de un formato de texto a otro.

Además de especificar un tipo de datos en una instrucción de declaración, puede forzar el tipo de datos de algunos elementos de programación mediante un carácter de tipo. Consulte [Caracteres de tipo](#).

Consumo de memoria

Al declarar un tipo de datos elemental, no se puede suponer con seguridad que su consumo de memoria es igual que su asignación de almacenamiento nominal. Esto se debe a las consideraciones siguientes:

- **Asignación de almacenamiento.** Common Language Runtime puede asignar almacenamiento en función de las características actuales de la plataforma en la que se ejecuta la aplicación. Si la memoria está casi llena, podría empaquetar los elementos declarados lo más juntos posible. En otros casos, podría alinear sus direcciones de memoria con límites de hardware naturales para optimizar el rendimiento.
- **Ancho de la plataforma.** La asignación de almacenamiento en una plataforma de 64 bits es diferente de la asignación en una plataforma de 32 bits.

Tipos de datos compuestos

Las mismas consideraciones se aplican a cada miembro de un tipo de datos compuesto, como una estructura o una matriz. No puede basarse simplemente en agregar las asignaciones de almacenamiento nominales de los miembros del tipo. Además, hay otras consideraciones, como las siguientes:

- **Sobrecarga.** Algunos tipos compuestos tienen requisitos de memoria adicionales. Por ejemplo, una matriz usa memoria adicional para la propia matriz y también para cada dimensión. En una plataforma de 32 bits, esta sobrecarga es actualmente de 12 bytes más 8 bytes por cada dimensión. En una plataforma de 64 bits, este requisito se duplica.
- **Diseño del almacenamiento.** No puede suponerse que el orden de almacenamiento en memoria es el mismo que el orden de la declaración. Ni siquiera se pueden hacer suposiciones sobre la alineación de bytes, como un límite de 2 bytes o de 4 bytes. Si va a definir una clase o estructura y necesita controlar el diseño del almacenamiento de sus miembros, puede aplicar el atributo [StructLayoutAttribute](#) a la clase o estructura.

Sobrecarga por objeto.

Un elemento `Object` que hace referencia a cualquier tipo de datos elemental o compuesto usa 4 bytes además de los datos contenidos en el tipo de datos.

Consulte también

- [StrConv](#)
- [StructLayoutAttribute](#)
- [Type Conversion Functions](#)
- [Resumen de las conversiones](#)
- [Caracteres de tipo](#)
- [Uso eficiente de los tipos de datos](#)